R3.04 · Conception · BUT2

Adapter

et Façade

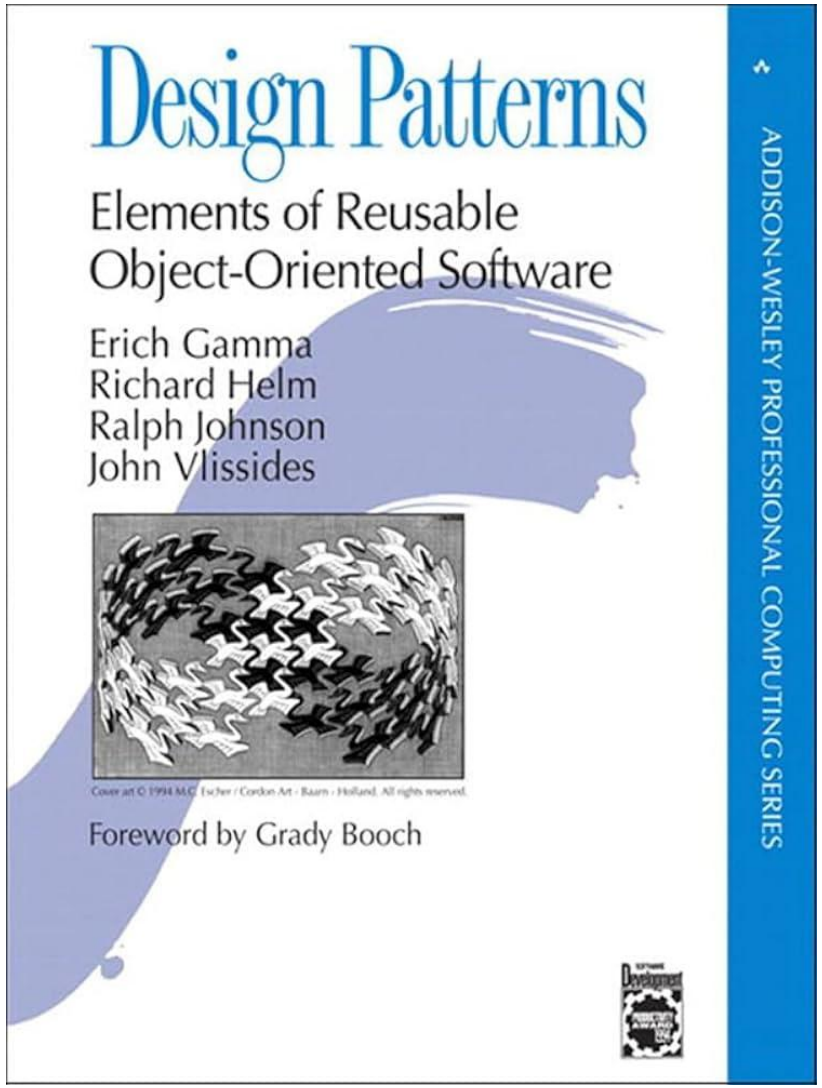
Deux patterns de structure du GoF

Deneuve Trystan – Diang Seydi – Cyprik Oscar

Sommaire

Partie
Patterns du GoF, métaphore, besoin, premier essai
Solution, généralisation, intention, rôles
Adapter et SOLID, limites
Façade et comparaison avec Adapter
Architecture hexagonale
Live coding (vidéo) et rétro-conception
QCM interactif
Bibliographie et IAgraphie

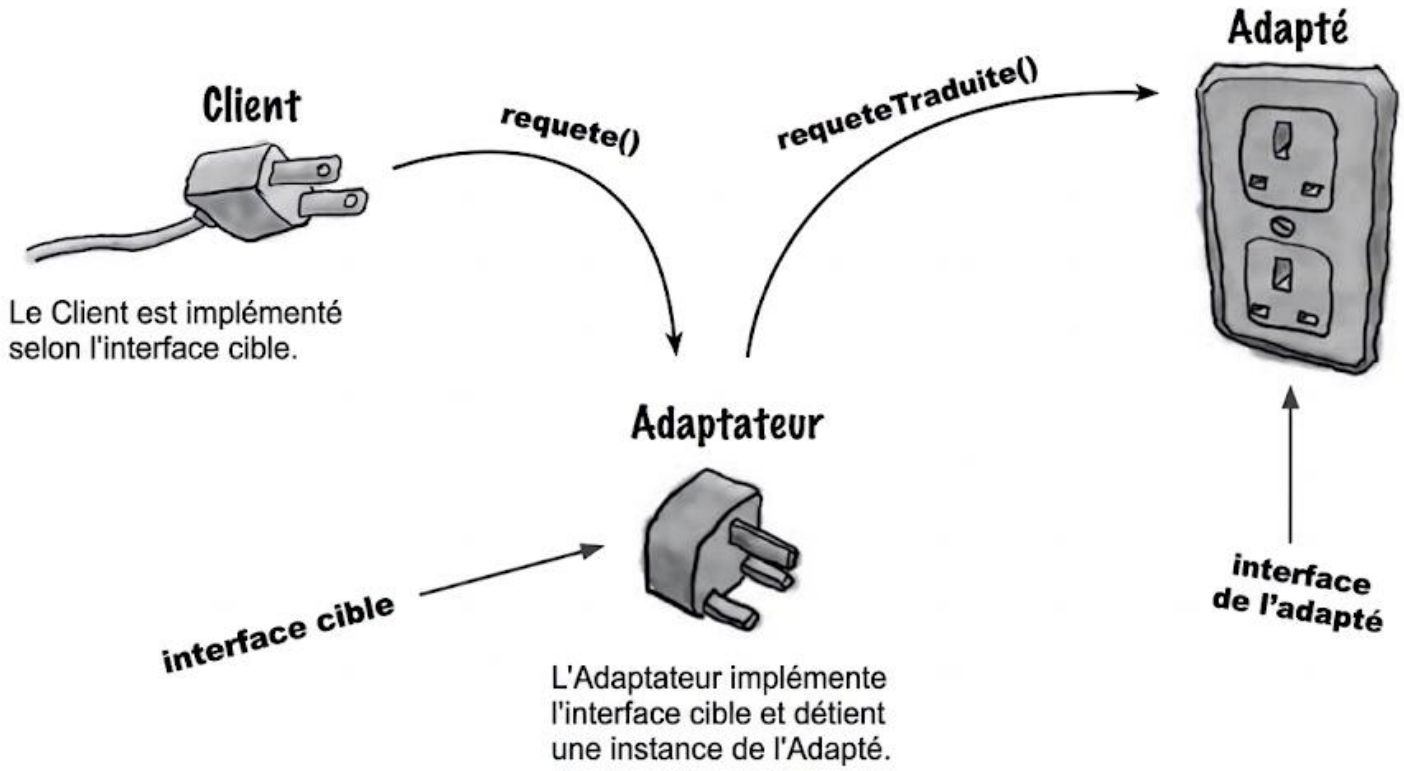
Gof



Création 5	Structure 7	Comportement 11

Les patterns Adapter et Façade se placent dans le bloc Structurel

Un chargeur français, une prise anglaise



Ni le chargeur ni la prise ne sont modifiés : tout se passe dans l'adaptateur.

Diagramme de classe et séquence génériques

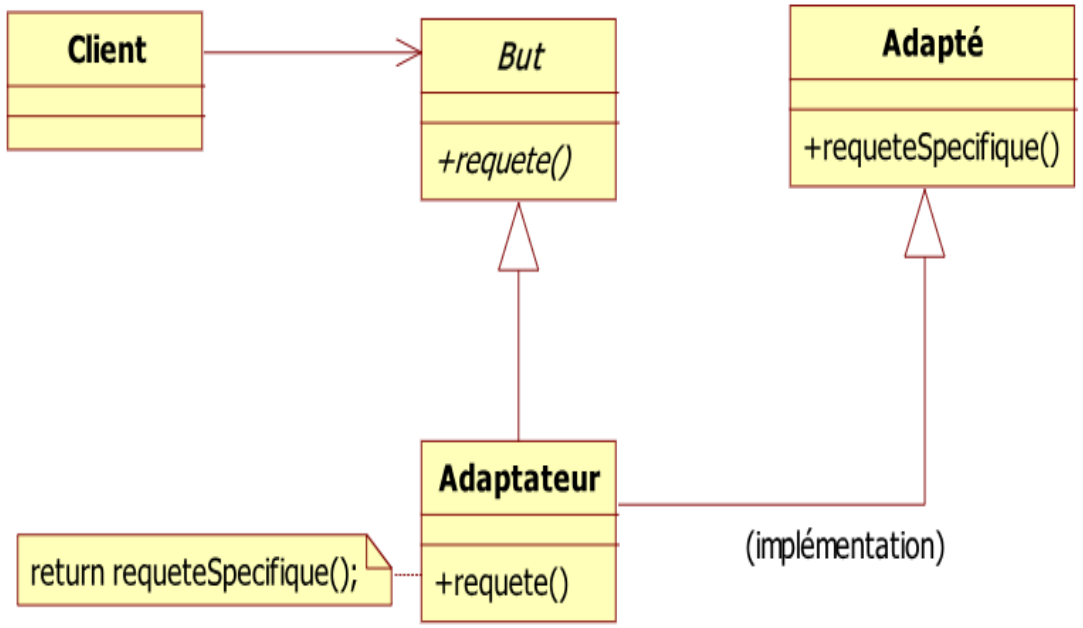


Diagramme de classe

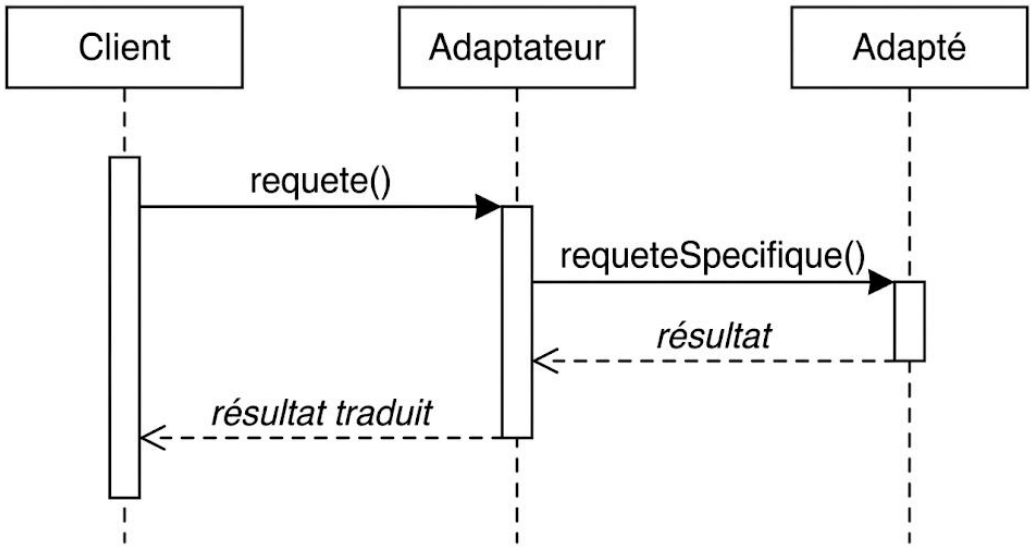


Diagramme de séquence

Encaisser avec la banque d'un prestataire

Ce que notre application attend

```
public interface Paiement {
    boolean payer(double euros, String carte);
}
```

Ce que le prestataire fournit

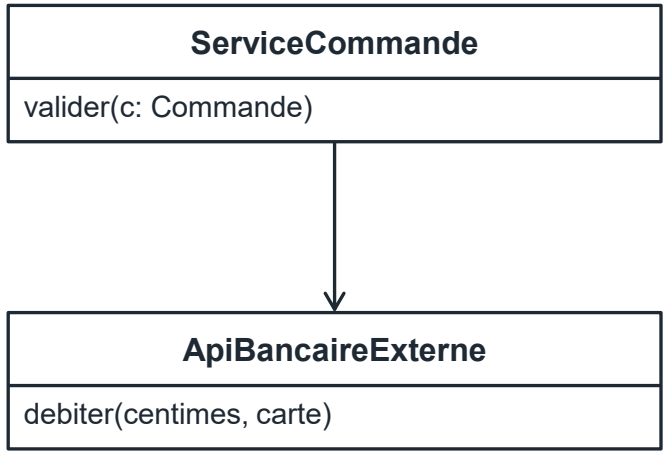
```
// Fournie par le prestataire (JAR), non modifiable
public class ApiBancaireExterne {
    // renvoie 0 si accepté, un code d'erreur sinon
    public int debiter(long centimes, String carte) {
        ...
    }
}
```

Écart	Paiement	ApiBancaireExterne
Nom de la méthode	payer	debiter
Unité du montant	euros (double)	centimes (long)
Résultat	boolean	code entier, 0 = accepté

Appeler la banque directement depuis le service

```
public class ServiceCommande {
    private final ApiBancaireExterne banque =
            new ApiBancaireExterne();

    public void valider(Commande c) {
        long centimes = Math.round(c.total() * 100);
        int code = banque.debiter(centimes, c.carte());
        if (code != 0) {
            throw new PaiementRefuse(code);
        }
    }
}
```



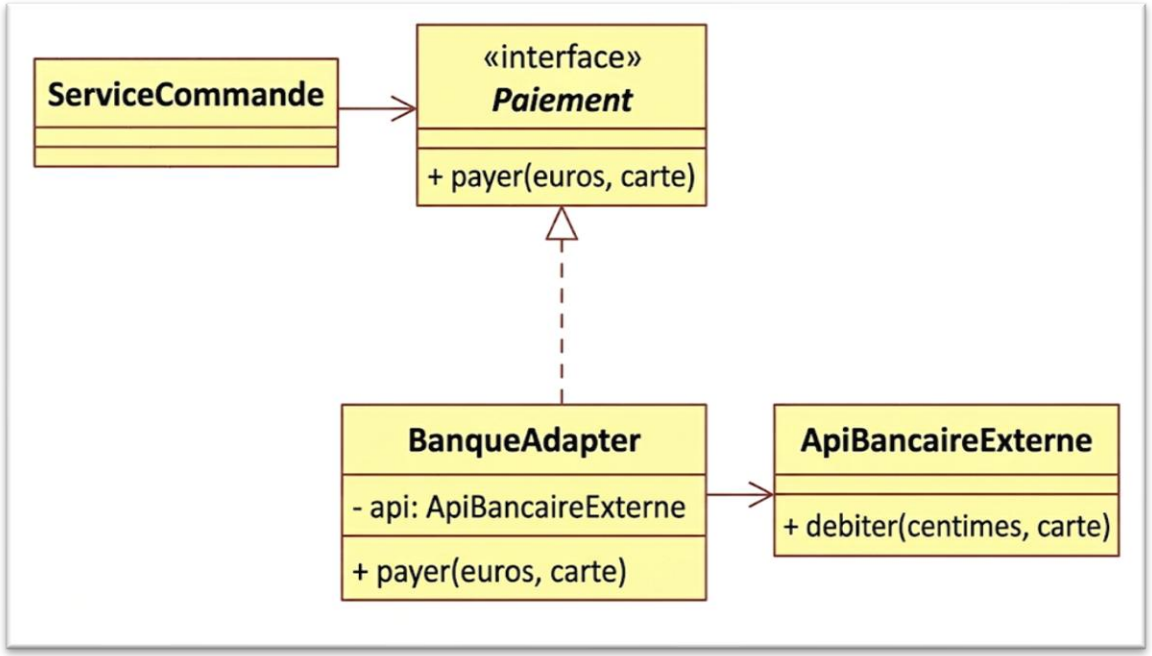


Personne ne l'utilise

Ce qui coince dans ce premier essai

Responsabilités mélangées	ServiceCommande convertit les euros en centimes et décode le code retour. Ce n'est pas son métier.
Fermé à l'évolution	Un second prestataire oblige à rouvrir valider() pour ajouter un if/else.
Difficile à tester	Impossible de tester valider() sans appeler la vraie API bancaire.
Contrat ignoré	Paiement existe mais le service dépend directement d'une classe concrète du prestataire.

Traduire dans une classe à part




```
public class BanqueAdapter implements Paiement {
    private final ApiBancaireExterne api;

    public BanqueAdapter(ApiBancaireExterne api) {
        this.api = api;
    }

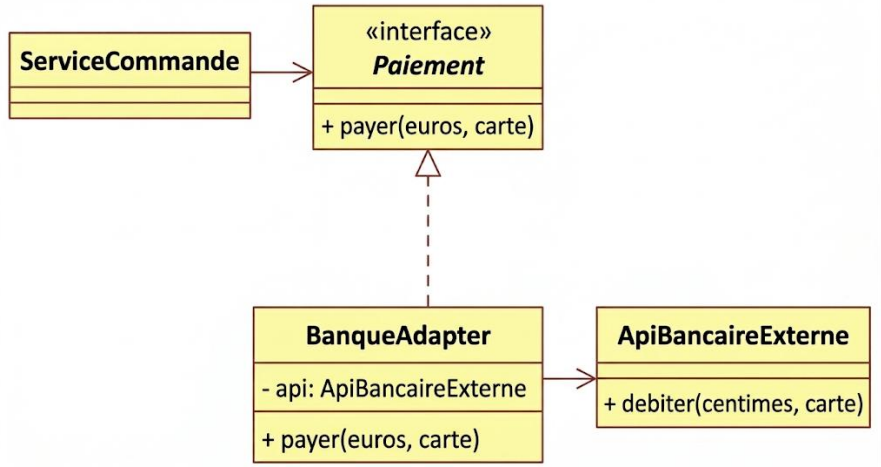
    @Override
    public boolean payer(double euros, String carte) {
        long centimes = Math.round(euros * 100);
        return api.debiter(centimes, carte) == 0;
    }
}
```

Branchement

```
Paiement p = new BanqueAdapter(new ApiBancaireExterne());
new ServiceCommande(p).valider(commande);
```

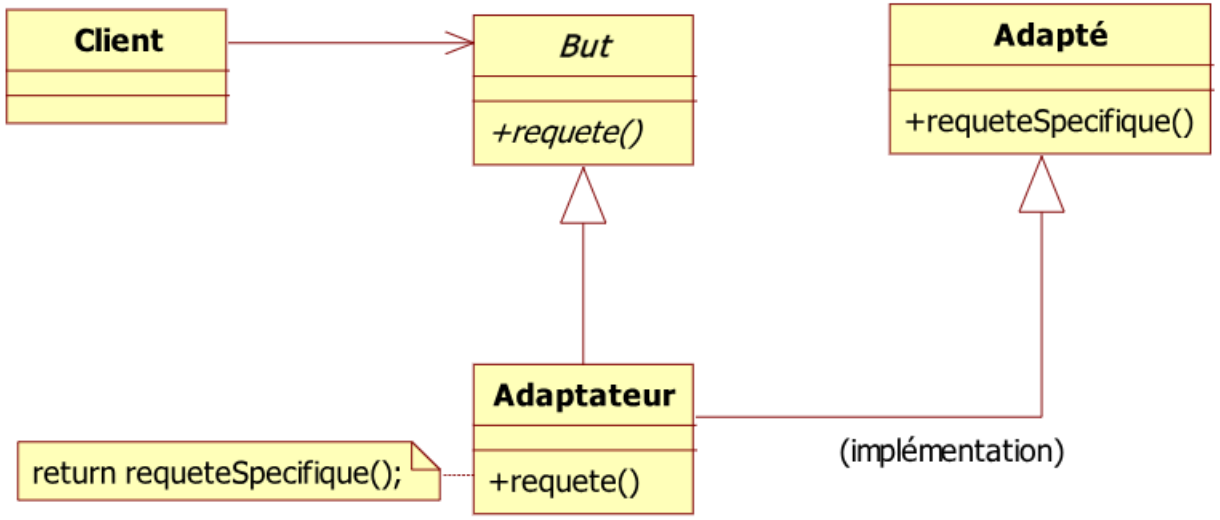
On remplace les noms métier par des rôles

Notre solution



Dans notre exemple	Rôle
ServiceCommande	Client
Paiement	Cible (Target)
BanqueAdapter	Adaptateur (Adapter)
ApiBancaireExterne	Adapté (Adaptee)

Version générique



Le diagramme du pattern, c'est celui de l'exemple une fois les noms métier effacés.

Adapter (Adaptateur) : nom, type, intention

Adapter

Aussi appelé Wrapper (enveloppe)
Type : pattern de structure du GoF

PROBLÉMATIQUE

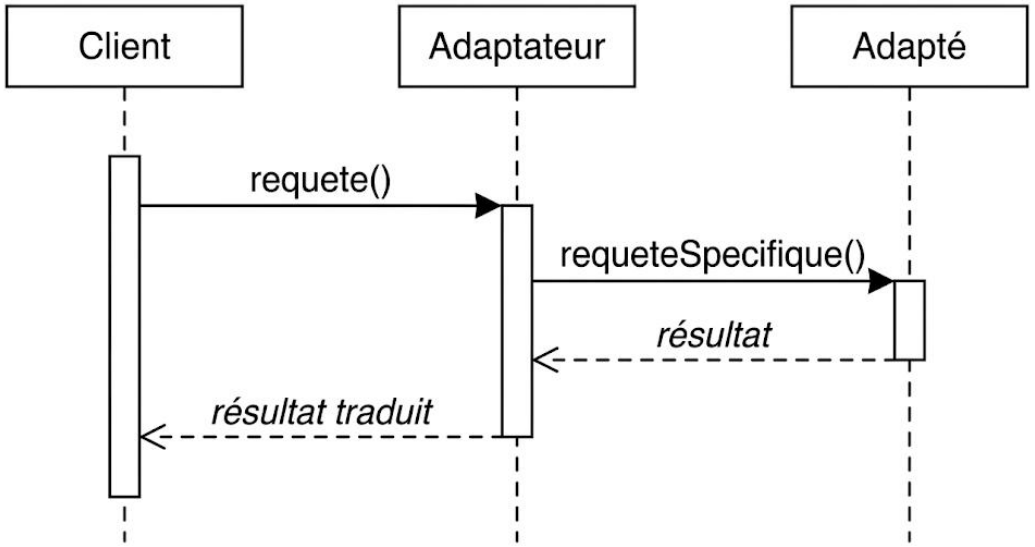
Comment faire collaborer une classe cliente et une classe existante dont les interfaces ne correspondent pas, sans modifier ni l'une ni l'autre ?

INTENTION

Convertir l'interface d'une classe en l'interface attendue par le client, pour que des classes aux interfaces incompatibles puissent travailler ensemble.

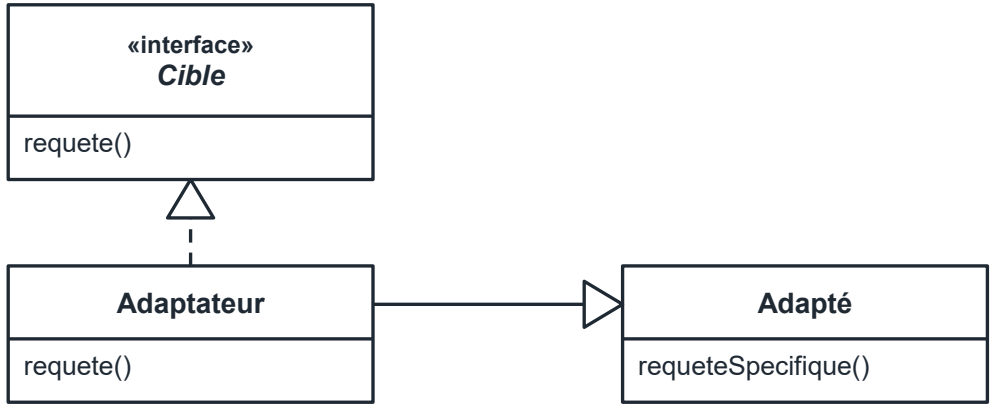
La solution du GoF : séquence et variante

Diagramme de séquence



Le client n'émet qu'un appel. La traduction (noms, unités, types) se fait dans l'adaptateur.

Variante : adaptateur de classe



L'adaptateur hérite de l'Adapté au lieu de le détenir. Possible en C++ (héritage multiple). En Java, impossible si la Cible est une classe, et inutilisable si l'Adapté est final. On préfère l'adaptateur d'objet (composition), celui de nos exemples.

Rôle des classes participantes

Classe générique	Description
Client	Utilise uniquement l'interface Cible. Il ignore qu'un adaptateur existe.
Cible (Target)	L'interface attendue par le client : le contrat qu'on veut conserver.
Adapté (Adaptee)	Classe existante, utile mais à l'interface incompatible. Bibliothèque ou code ancien, elle reste inchangée.
Adaptateur (Adapter)	Implémente la Cible et détient une référence vers l'Adapté. Chaque appel reçu est traduit : noms, unités, types, erreurs.

Adapter et SOLID

Principe	Ce que le pattern apporte
S · Single responsibility principle	La traduction euros/centimes et code/boolean vit dans BanqueAdapter. ServiceCommande ne gère que la commande.
O · Open/Close principle	Un second prestataire = une nouvelle classe Adapter. ServiceCommande n'est pas rouvert.
L · Liskov Substitution principle	Tout Adapter peut remplacer un autre Paiement, à condition de respecter le contrat : mêmes exceptions, même sens du résultat.
I · Interface segregation principle	La Cible reste étroite, écrite pour le client. On n'expose pas toutes les méthodes de l'API externe.
D · Dependency inversion principle	ServiceCommande dépend de l'abstraction Paiement, pas de la bibliothèque du prestataire.

O et D sont les deux principes que le pattern sert le plus directement. Maintenabilité : si le prestataire change son API, on modifie BanqueAdapter et rien d'autre.

ADAPTER · LIMITES

Un appel au lieu de quatre

Sans façade

```
// à chaque endroit où l'on passe une commande
stock.reserver(c);
paiement.payer(c.total(), c.carte());
livraison.planifier(c);
facturation.emettre(c);
```

Le contrôleur web, l'appli mobile et le traitement par lots recopient ces lignes. L'ordre compte (payer avant d'expédier) et chaque appelant doit le connaître.

Avec façade

```
public class CommandeFacade {
    private final Stock stock;
    private final Paiement paiement;
    private final Livraison livraison;
    private final Facturation facturation;

    public void passer(Commande c) {
        stock.reserver(c);
        paiement.payer(c.total(), c.carte());
        livraison.planifier(c);
        facturation.emettre(c);
    }
}
// côté appelant :
commandeFacade.passer(commande);
```

Façade : intention et structure

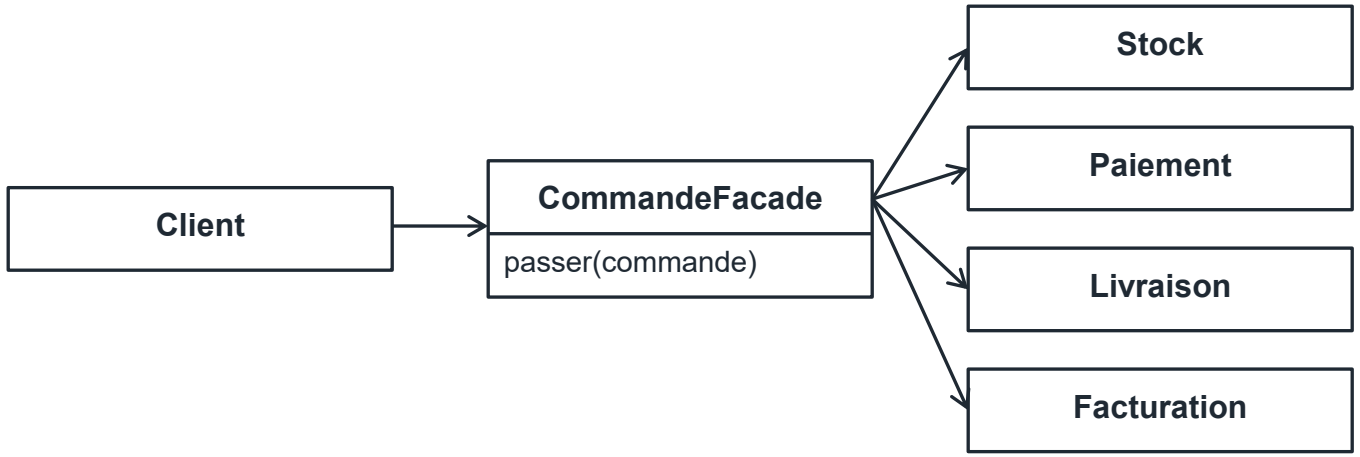
PROBLÉMATIQUE

Comment utiliser un sous-système de plusieurs classes sans que chaque client en connaisse le détail et l'ordre d'appel ?

INTENTION

Fournir une interface unique et simplifiée devant un ensemble d'interfaces d'un sous-système, pour le rendre plus simple à utiliser.

Type : pattern de structure

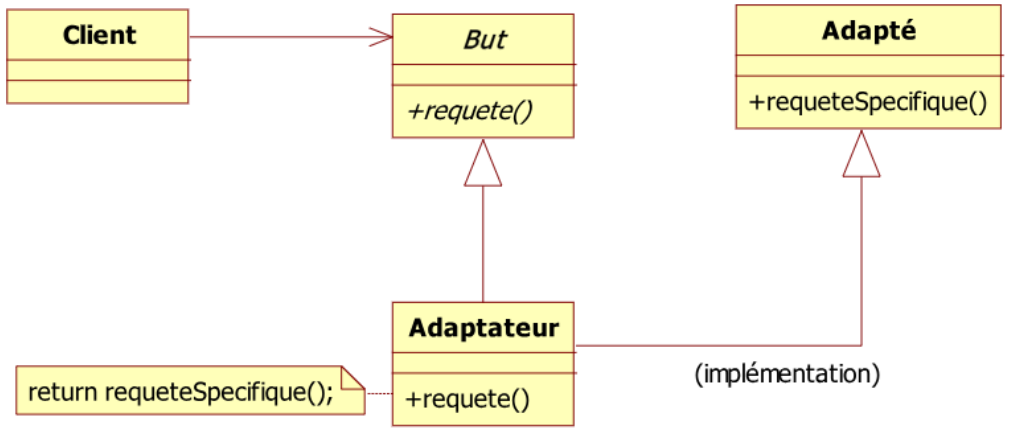


Les sous-systèmes restent utilisables directement : la façade simplifie, elle n'interdit rien.

COMPARAISON

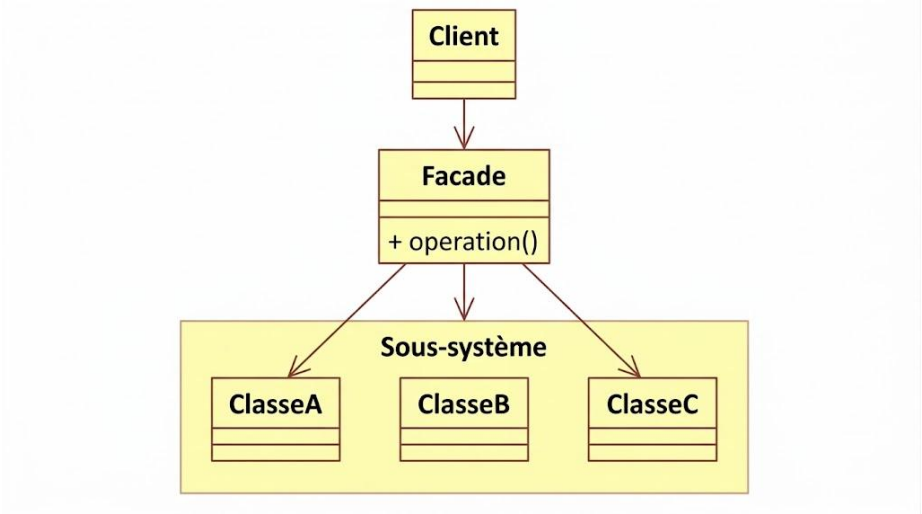
Adapter ou Façade ?

Adapter



Intention : convertir l'interface d'une classe en celle qu'attend le client.

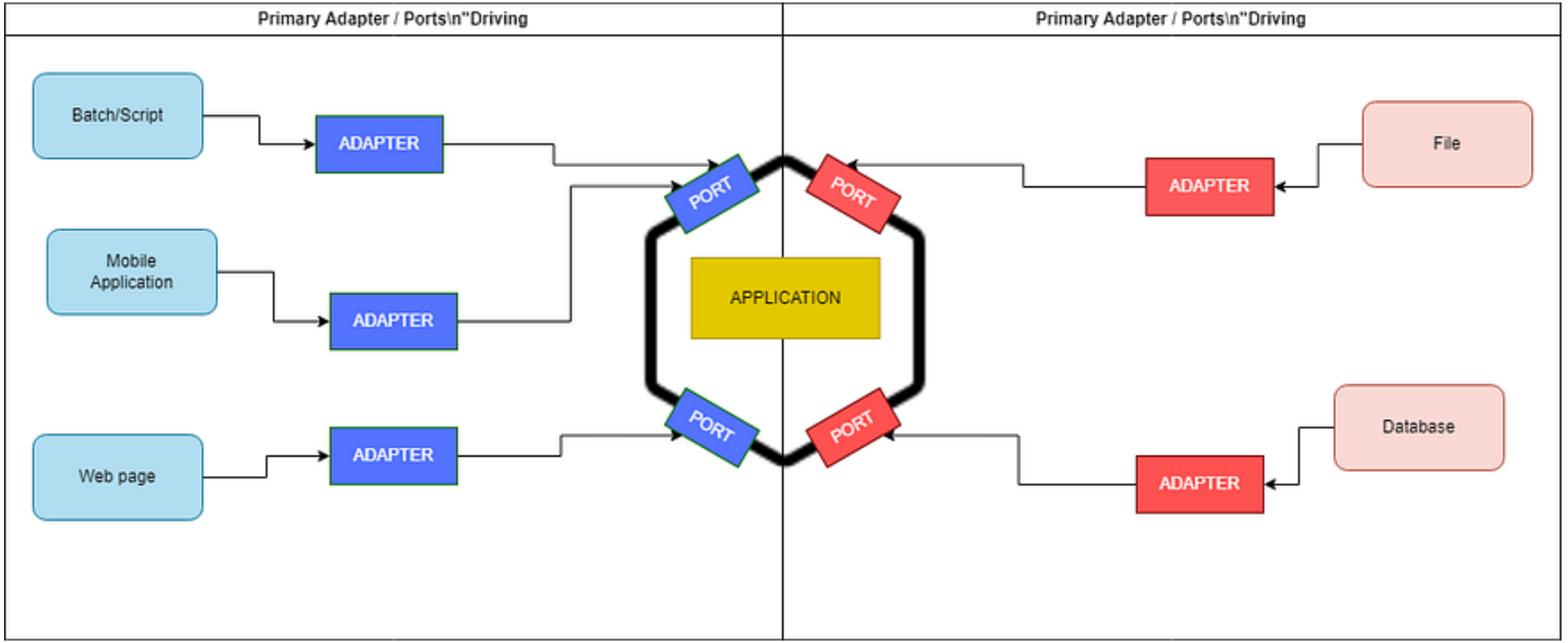
Façade



Intention : fournir une interface simplifiée à un ensemble d'interfaces d'un sous-système.

	Adapter	Façade
Interface visée	Elle existe déjà, on s'y conforme	Elle n'existe pas, on la crée
Classes derrière	Une, en général	Tout un sous-système
But	Faire coopérer des interfaces incompatibles	Simplifier l'usage

Des adaptateurs de chaque côté du domaine



https://medium.com/@erickzanetti/understanding-hexagonal-architecture-ports-and-adapters-8945fc3e3dc0

Manette et clavier dans un jeu

```java
public class Personnage {
    private final String nom;
    private final Controle controle;

    public Personnage(String nom, Controle controle) {
        this.nom = nom;
        this.controle = controle;
    }

    public void mettreAJour() {
        double dir = controle.direction();
        boolean saut = controle.saut();

        System.out.printf("[%s] Direction: %.2f | Action: %s%n",
                nom, dir, saut ? "SAUT !" : "Au sol");
    }
}
```

```java
public class ManetteAdapter implements Controle {
    private final ManetteBluetooth manette;

    public ManetteAdapter(ManetteBluetooth manette) {
        this.manette = manette;
    }

    @Override
    public double direction() {
        return manette.axeX() / 32768.0;
    }

    @Override
    public boolean saut() {
        return manette.boutonA();
    }
}
```

```java
public interface Controle {
    /**
     * @return Direction entre -1.0 (gauche) et 1.0 (droite), 0.0 au repos
     */
    double direction();

    /**
     * @return true si le bouton de saut est pressé
     */
    boolean saut();
}
```

```java
public class ManetteBluetooth {
    private int positionStickX;
    private boolean boutonAPresse;

    public ManetteBluetooth() {
        this.positionStickX = 0;
        this.boutonAPresse = false;
    }

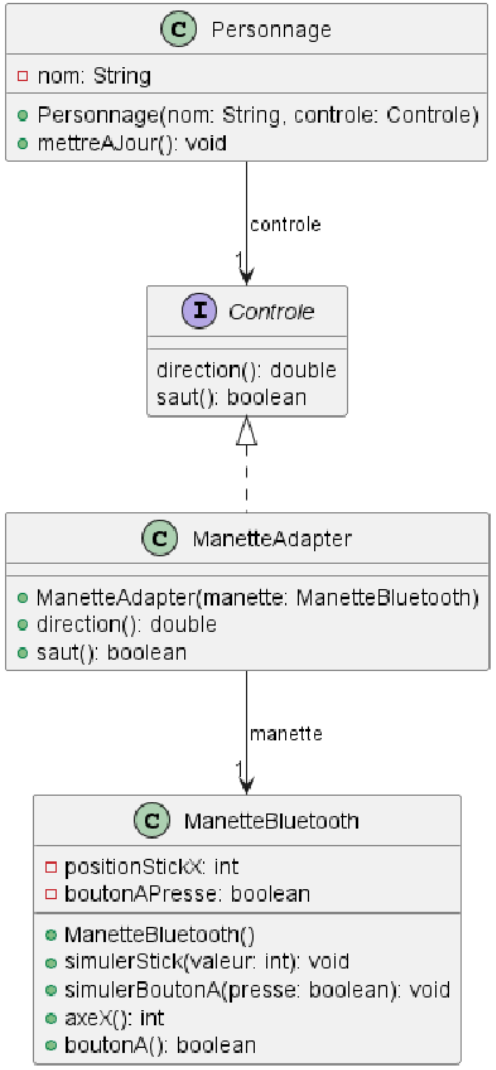
    // Méthodes pour simuler l'état physique du matériel
    public void simulerStick(int valeur) {
        this.positionStickX = valeur;
    }

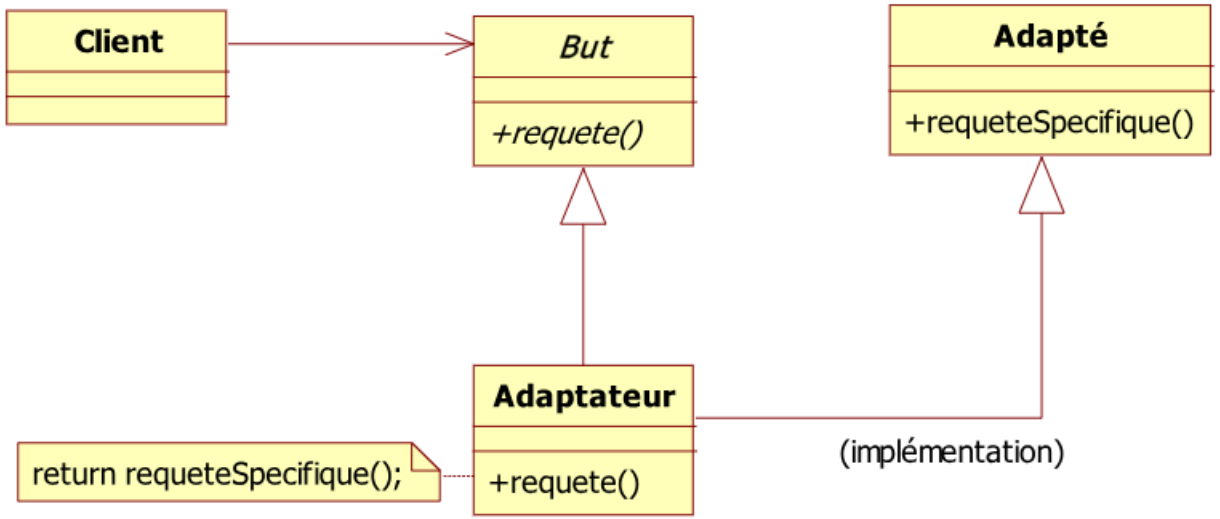
    public void simulerBoutonA(boolean presse) {
        this.boutonAPresse = presse;
    }

    /**
     * @return Valeur brute de l'axe X entre -32768 et 32767
     */
    public int axeX() {
        return positionStickX;
    }

    /**
     * @return État brut du bouton A
     */
    public boolean boutonA() {
        return boutonAPresse;
    }
}
```

Comparaison du diagramme obtenu avec celui générique

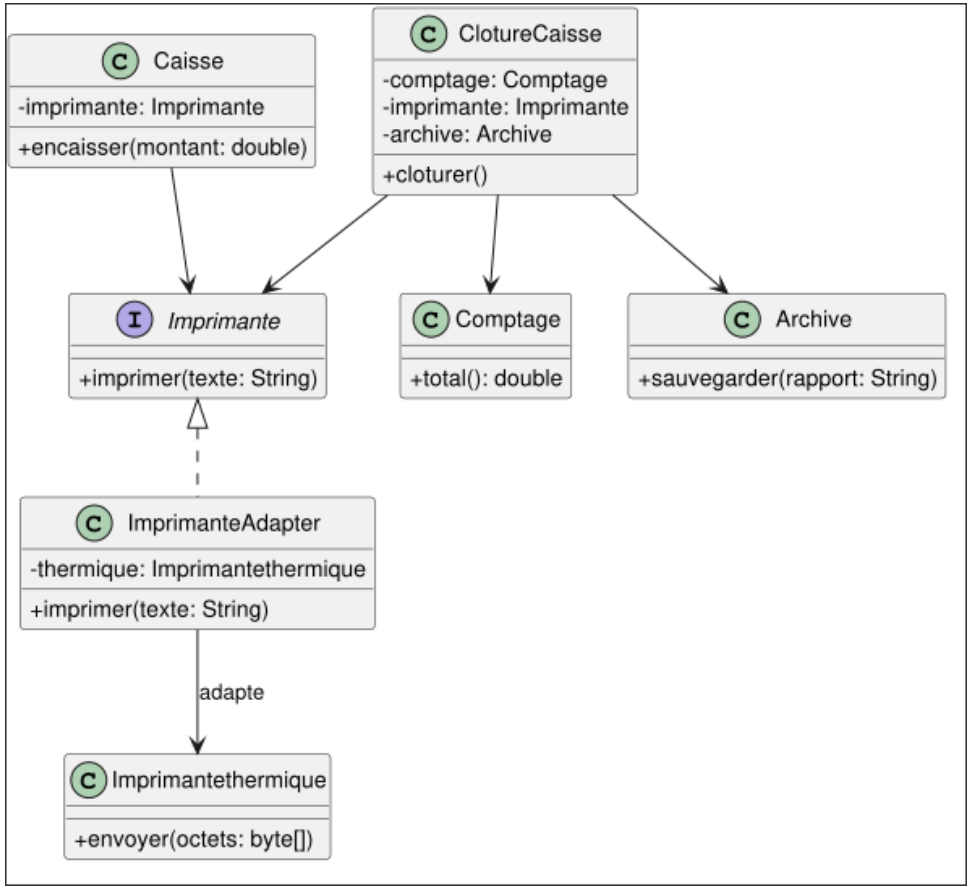




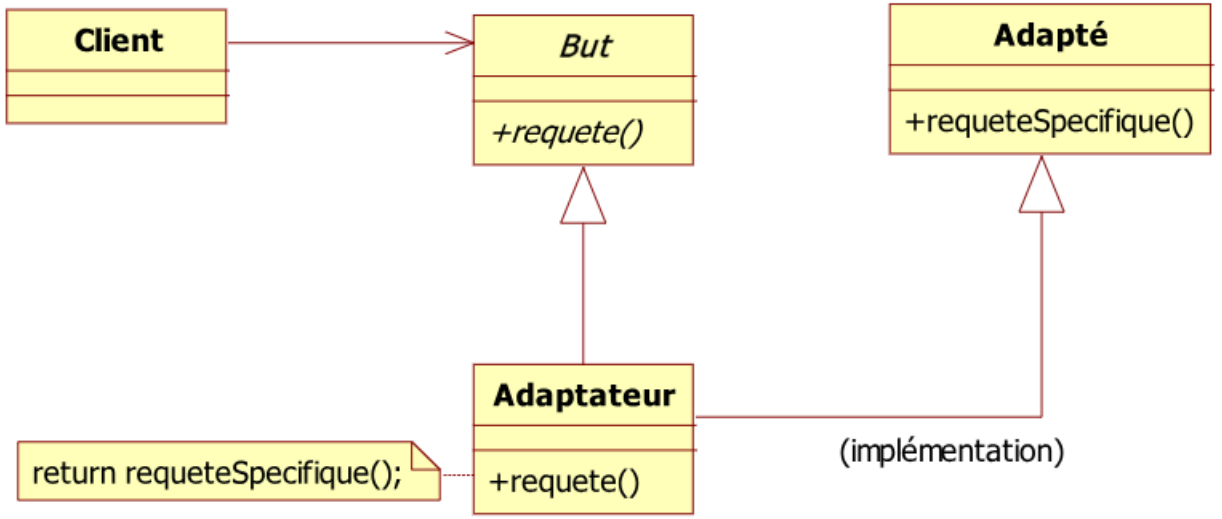
LIVE CODING

Notre code face au pattern

Adapter : diagramme obtenu par rétro-conception



Adapter : diagramme générique



Bibliographie et webographie

https://refactoring.guru/design-patterns/adapter
https://refactoring.guru/design-patterns/facade
https://aakash01.github.io/design%20patterns/2016/06/04/Facade-Pattern
https://aakash01.github.io/design%20patterns/2016/06/04/Adapter-Pattern

Usage de l'IA dans ce travail

Outil	Nature de l'apport	Prompt de synthèse
Gemini	Comprendre correctement le pattern	explique moi le pattern Adapter en java
Claude	Comprendre avec une autre IA le pattern	explique moi le pattern Adapter en java